

Metodi

Metodi

Come definire e usare i metodi nelle classi C++.

Cosa sono i metodi?

Un oggetto non è solo un contenitore di dati — può anche fare cose. I **metodi** sono le azioni che un oggetto sa compiere.

Un oggetto `Cane` ha dati (nome, razza, età) ma può anche abbaiare, mangiare, correre. Un oggetto `ContoBancario` ha un saldo, ma può anche ricevere depositi e prelievi.

I metodi sono semplicemente **funzioni scritte dentro una classe**.

Definire metodi dentro la classe

```
class Cerchio {
public:
    double raggio;

    // Metodo che calcola l'area
    double area() {
        return 3.14159 * raggio * raggio;
    }

    // Metodo che calcola il perimetro
    double perimetro() {
        return 2 * 3.14159 * raggio;
    }
};

int main() {
    Cerchio c;
    c.raggio = 5.0;

    cout << "Area: " << c.area() << endl;          // usa automaticamente
c.raggio
    cout << "Perimetro: " << c.perimetro() << endl;
}
```

Il metodo ha accesso diretto ai dati dell'oggetto su cui viene chiamato.

Separare dichiarazione e definizione

Per classi grandi, puoi dichiarare il metodo dentro la classe e scriverne il corpo fuori, usando `NomeClasse::` :

```

class Cerchio {
public:
    double raggio;

    double area();          // solo la firma (dichiarazione)
    double perimetro();     // solo la firma
};

// Il corpo viene scritto fuori dalla classe
double Cerchio::area() {
    return 3.14159 * raggio * raggio;
}

double Cerchio::perimetro() {
    return 2 * 3.14159 * raggio;
}

```

L'operatore `::` dice "questo metodo appartiene alla classe Cerchio".

Metodi che non modificano i dati: **const**

Se un metodo legge solo i dati senza cambiarli, aggiungere `const` alla fine è una buona abitudine. Serve come promessa al compilatore (e al lettore del codice):

```

class Rettangolo {
public:
    double base;
    double altezza;

    double area() const {          // const = non modifica i dati
        return base * altezza;
    }

    void ridimensiona(double b, double a) { // nessun const = può
        modificare
        base = b;
        altezza = a;
    }
};

```

Se provi a modificare un campo dentro un metodo `const`, il compilatore ti segnala l'errore.

Risolvere conflitti di nome con `this`

Dentro un metodo, `this` è un puntatore speciale che indica "l'oggetto corrente". Si usa raramente, ma è utile quando il nome di un parametro coincide con il nome di un campo:

```
class Persona {  
public:  
    string nome;  
  
    void setName(string nome) {  
        // "nome" da solo si riferisce al parametro  
        // "this->nome" si riferisce al campo della classe  
        this->nome = nome;  
    }  
};
```

Getter e Setter: porte controllate per i dati privati

Quando i dati di una classe sono privati (non accessibili dall'esterno), si usano metodi appositi per leggerli e modificarli:

- **Getter:** legge un dato privato
- **Setter:** modifica un dato privato (può anche validare)

```

class Temperatura {
private:
    double gradi;    // non accessibile dall'esterno

public:
    // Setter: imposta la temperatura, ma solo se valida
    void setGradi(double t) {
        if (t >= -273.15) {    // non può scendere sotto lo zero assoluto
            gradi = t;
        } else {
            cout << "Temperatura non valida!" << endl;
        }
    }

    // Getter: legge la temperatura
    double getGradi() const {
        return gradi;
    }

    // Metodo utile: converte in Fahrenheit
    double inFahrenheit() const {
        return gradi * 9.0/5.0 + 32;
    }
};

int main() {
    Temperatura t;
    t.setGradi(100.0);
    cout << t.getGradi() << "°C = " << t.inFahrenheit() << "°F" << endl;
    // 100°C = 212°F

    t.setGradi(-300.0);    // Temperatura non valida!
    return 0;
}

```

Esempio pratico: conto bancario

```
#include <iostream>
#include <string>
using namespace std;

class ContoBancario {
private:
    string intestatario;
    double saldo;

public:
    // Inizializza il conto
    void apri(string nome, double saldoIniziale) {
        intestatario = nome;
        saldo = saldoIniziale;
    }

    // Aggiunge denaro al saldo
    void deposita(double importo) {
        if (importo > 0) {
            saldo += importo;
            cout << "Depositati " << importo
                 << " euro. Saldo: " << saldo << " euro." << endl;
        }
    }

    // Toglie denaro dal saldo (solo se ci sono fondi)
    bool preleva(double importo) {
        if (importo > 0 && importo <= saldo) {
            saldo -= importo;
            cout << "Prelevati " << importo
                 << " euro. Saldo: " << saldo << " euro." << endl;
            return true;
        }
        cout << "Fondi insufficienti!" << endl;
        return false;
    }

    // Getter per leggere i dati privati
    double getSaldo() const { return saldo; }
    string getIntestatario() const { return intestatario; }
};

int main() {
    ContoBancario conto;
```

```
conto.apri("Alice", 1000.0);

conto.deposita(500.0);    // Depositati 500 euro. Saldo: 1500.
conto.preleva(200.0);    // Prelevati 200 euro. Saldo: 1300.
conto.preleva(2000.0);   // Fondi insufficienti!

cout << "Saldo finale: " << conto.getSaldo() << " euro" << endl;

return 0;
}
```